

Laboratório — Transações, ACID e 2PC no iximiuz Labs

Disciplina: Banco de Dados II — 2026.2 · **Aula 07** · **Prof. Alysson M. Bruno**

Objetivo

Controlar transações no PostgreSQL 17 (BEGIN, COMMIT, ROLLBACK e SAVEPOINT), observar na prática cada uma das propriedades ACID e executar à mão o protocolo de confirmação em duas fases (2PC) entre dois bancos.

Antes de começar: leia esta página

Este laboratório é **para fazer em casa**, no seu tempo. Ele está dividido em **sete atividades independentes**, e nenhuma leva mais de **20 minutos**.

Por que sete atividades pequenas

A conta gratuita do iximiuz Labs dá **cerca de uma hora de ambiente por dia**, e ao encerrar a sessão a máquina é **destruída**: nada do que você digitou fica guardado.

Por isso **cada atividade começa do zero e não depende de nenhuma outra**. Faça uma, duas ou três por dia, na ordem que quiser. Todas começam com os mesmos três comandos de preparo, que levam cerca de um minuto.

Para não desperdiçar a sua hora:

1. **leia a atividade inteira antes** de iniciar a máquina;
2. **anote** cada resultado — o ambiente você perde, a anotação fica;
3. não tenha receio de fechar: reabrir custa um minuto.

Atividade	O que você vai ver	Tempo	Terminais
1	Tudo ou nada: BEGIN, COMMIT, ROLLBACK e a transação abortada	15 min	1
2	SAVEPOINT: desfazer só um pedaço do lote	15 min	1
3	Isolamento: o que a outra sessão enxerga	15 min	2
4	Durabilidade: derrubar o servidor e ver o WAL trabalhar	15 min	2
5	2PC: as duas fases e o voto “não”	20 min	2
6	2PC: o coordenador caiu	15 min	2
7	A transação pelo pgAdmin	15 min	1 + navegador

Como abrir o ambiente

- Entre na sua conta do iximiuz Labs e, na primeira vez, **matricule-se na turma da disciplina:** <https://labs.iximiuz.com/trainings/aulas-bd2-2026-2-51d1c714>, botão **Enroll**. Vale para o semestre inteiro.
- Abra <https://labs.iximiuz.com/playgrounds/BD2-AULA07-ad3d665d> e clique em **Start Playground**. O PostgreSQL 17 e o pgAdmin já vêm baixados, e os arquivos do laboratório já estão em ~/lab07.
- O playground tem quatro abas:

Aba	Para quê
Sessão 1	o primeiro terminal
Sessão 2	um segundo terminal, na mesma máquina (atividades 3 a 6)
IDE	editor, para ler os arquivos com calma
pgAdmin	abre a interface web do pgAdmin em outra janela (atividade 7)

- Para colar um comando no terminal, use **Ctrl + Shift + V** ou o botão direito do mouse.
- Se estiver somente no celular, acompanhe cada atividade pelos slides e anote os resultados esperados que aparecem aqui.

Os arquivos que já estão na máquina

Arquivo em ~/lab07	Para que serve
compose.yaml	sobe dois contêineres: o PostgreSQL 17 (bd2-aula07-pg) e o pgAdmin (bd2-aula07-pgadmin)
preparar.sql	deixa os bancos no ponto de partida; pode ser rodado quantas vezes quiser
servers.json	cadastra o servidor no pgAdmin, para você não ter de digitar nada

O cenário: a Cooperativa Cerrado

Uma cooperativa de crédito com agências em **Palmas** e em **Gurupi**. O preparar.sql cria três bancos de dados:

Banco	Tabela conta	Usado nas atividades
banco	Ana, Bruno, Carla e Davi — as quatro contas	1, 2, 3, 4 e 7
agencia_palmas	só Ana e Bruno	5 e 6
agencia_gurupi	só Carla e Davi	5 e 6

Saldos iniciais: **Ana R\$ 1.000,00 · Bruno R\$ 500,00 · Carla R\$ 200,00 · Davi R\$ 50,00** — total de **R\$ 1.750,00**. A coluna saldo tem a restrição CHECK (saldo >= 0): **nenhuma conta pode ficar negativa**.

O preparo — igual em todas as atividades

Na aba **Sessão 1**:

```
cd ~/lab07
podman compose up -d
podman exec -i bd2-aula07-pg psql -U aluno -d banco < preparar.sql
```

Resultado esperado: o compose up imprime os nomes bd2-aula07-pg e bd2-aula07-pgadmin e volta ao prompt em poucos segundos. O preparar.sql termina com:

```
Laboratório da Aula 07 pronto. Contas no banco "banco":
 id | titular | cidade |  saldo
-----+-----+-----+-----
  1 | Ana     | Palmas | 1000.00
  2 | Bruno   | Palmas |  500.00
  3 | Carla   | Gurupi |  200.00
  4 | Davi    | Gurupi |   50.00
(4 rows)
```

Se aparecer connection refused ou the database system is starting up, espere dez

segundos e repita só a última linha.

As senhas deste laboratório (aluno) são apenas didáticas. Nunca reutilize senhas de exemplo em sistemas reais.

Atividade 1 — Tudo ou nada

Tempo: 15 minutos · **Terminais:** 1 · **Propriedades:** Atomicidade e Consistência

Objetivo: ver que as operações de uma transação são confirmadas juntas ou desfeitas juntas — inclusive quando um erro acontece no meio.

1.1 — Preparar e entrar no banco

Faça o preparo (os três comandos acima) e entre no cliente:

```
podman exec -it bd2-aula07-pg psql -U aluno -d banco
```

Resultado esperado: o prompt muda para banco=#.

Confira o total de dinheiro da cooperativa. Guarde este número: nenhuma transferência deveria mudá-lo.

```
SELECT sum(saldo) AS total FROM conta;
```

Resultado esperado: 1750.00.

1.2 — Uma transferência confirmada

Ana transfere R\$ 100,00 para Bruno. São duas operações, e elas precisam acontecer juntas:

```
BEGIN;
UPDATE conta SET saldo = saldo - 100 WHERE id = 1;    -- débito em Ana
UPDATE conta SET saldo = saldo + 100 WHERE id = 2;    -- crédito em Bruno
COMMIT;

SELECT id, titular, saldo FROM conta ORDER BY id;
```

Resultado esperado: Ana 900.00, Bruno 600.00, Carla 200.00, Davi 50.00.

1.3 — Desistir no meio: ROLLBACK

Bruno começa a transferir R\$ 50,00 para Carla, mas desiste:

```
BEGIN;  
UPDATE conta SET saldo = saldo - 50 WHERE id = 2;  
UPDATE conta SET saldo = saldo + 50 WHERE id = 3;  
SELECT id, titular, saldo FROM conta ORDER BY id;
```

Resultado esperado: dentro da transação você já vê Bruno 550.00 e Carla 250.00.

```
ROLLBACK;  
SELECT id, titular, saldo FROM conta ORDER BY id;
```

Resultado esperado: Bruno volta a 600.00 e Carla a 200.00. As duas alterações foram desfeitas juntas.

1.4 — Um erro no meio: a transação abortada

Agora Davi tenta transferir R\$ 80,00 para Carla — mas Davi só tem R\$ 50,00. Repare que o crédito em Carla vem **antes** do débito em Davi:

```
BEGIN;  
UPDATE conta SET saldo = saldo + 80 WHERE id = 3;    -- crédito em Carla: funciona  
UPDATE conta SET saldo = saldo - 80 WHERE id = 4;    -- débito em Davi: falha
```

Resultado esperado: o primeiro UPDATE responde UPDATE 1; o segundo responde:

```
ERROR:  new row for relation "conta" violates check constraint  
"conta_saldo_check"  
DETAIL:  Failing row contains (4, Davi, Gurupi, -30.00).
```

Tente consultar:

```
SELECT id, titular, saldo FROM conta ORDER BY id;
```

Resultado esperado:

```
ERROR:  current transaction is aborted, commands ignored until end of transaction  
block
```

A transação está **abortada**: o servidor não aceita mais nada dela. Tente confirmar mesmo assim:

```
COMMIT;
```

Resultado esperado: o servidor responde **ROLLBACK**, e não COMMIT.

```
SELECT id, titular, saldo FROM conta ORDER BY id;
```

Resultado esperado: Carla continua com 200.00. O crédito de R\$ 80,00, que tinha funcionado, **também foi desfeito**. Isso é a **atomicidade**; e quem barrou a operação foi a restrição CHECK, que guarda a **consistência**.

1.5 — Sem BEGIN não há volta

Sem BEGIN, cada comando é uma transação sozinha, confirmada na hora (*autocommit*):

```
UPDATE conta SET saldo = saldo - 10 WHERE id = 1;  
ROLLBACK;
```

Resultado esperado:

```
UPDATE 1  
WARNING: there is no transaction in progress  
ROLLBACK
```

```
SELECT id, titular, saldo FROM conta ORDER BY id;  
SELECT sum(saldo) AS total FROM conta;
```

Resultado esperado: Ana ficou com 890.00 e o total caiu para 1740.00. Os R\$ 10,00 saíram e não entraram em lugar nenhum — o ROLLBACK chegou tarde, porque não havia transação para desfazer.

Saia com \q.

Anote: o que o servidor respondeu ao COMMIT da seção 1.4, e por quê?

Atividade 2 — SAVEPOINT: desfazer só um pedaço

Tempo: 15 minutos · **Terminais:** 1

Objetivo: num lote de pagamentos, desfazer apenas o pagamento que falhou, sem perder os que deram certo.

2.1 — Preparar e entrar no banco

Faça o preparo e entre no cliente:

```
podman exec -it bd2-aula07-pg psql -U aluno -d banco
```

2.2 — Um lote com um pagamento impossível

Ana vai pagar três pessoas numa única transação: R\$ 100,00 para Bruno, R\$ 2.000,00 para Davi (ela não tem esse dinheiro) e R\$ 50,00 para Carla.

```
BEGIN;

-- pagamento 1: Ana -> Bruno, R$ 100,00
UPDATE conta SET saldo = saldo - 100 WHERE id = 1;
UPDATE conta SET saldo = saldo + 100 WHERE id = 2;

SAVEPOINT pagou_bruno;          -- marca o ponto seguro

-- pagamento 2: Ana -> Davi, R$ 2.000,00
UPDATE conta SET saldo = saldo + 2000 WHERE id = 4;
UPDATE conta SET saldo = saldo - 2000 WHERE id = 1;
```

Resultado esperado: o último UPDATE falha:

```
ERROR:  new row for relation "conta" violates check constraint
"conta_saldo_check"
DETAIL:  Failing row contains (1, Ana, Palmas, -1100.00).
```

A transação está abortada — mas agora existe um ponto seguro para onde voltar.

2.3 — Voltar ao ponto seguro

```
ROLLBACK TO SAVEPOINT pagou_bruno;
SELECT id, titular, saldo FROM conta ORDER BY id;
```

Resultado esperado: o ROLLBACK TO SAVEPOINT responde ROLLBACK e a consulta **volta a funcionar:** Ana 900.00, Bruno 600.00, Carla 200.00, Davi 50.00. O pagamento 1 continua lá; só o pagamento 2 foi desfeito — os R\$ 2.000,00 que tinham entrado na conta de Davi saíram de novo.

2.4 — Seguir com o lote e confirmar

```
-- pagamento 3: Ana -> Carla, R$ 50,00
UPDATE conta SET saldo = saldo - 50 WHERE id = 1;
UPDATE conta SET saldo = saldo + 50 WHERE id = 3;

COMMIT;

SELECT id, titular, saldo FROM conta ORDER BY id;
SELECT sum(saldo) AS total FROM conta;
```

Resultado esperado: Ana 850.00, Bruno 600.00, Carla 250.00, Davi 50.00; total 1750.00. Dois pagamentos confirmados, um recusado, e o dinheiro da cooperativa intacto.

2.5 — RELEASE e o ROLLBACK que apaga tudo

```
BEGIN;  
UPDATE conta SET saldo = saldo - 30 WHERE id = 1;  
SAVEPOINT s1;  
UPDATE conta SET saldo = saldo + 30 WHERE id = 4;  
RELEASE SAVEPOINT s1;  
ROLLBACK TO SAVEPOINT s1;
```

Resultado esperado: RELEASE responde RELEASE; o ROLLBACK TO falha com ERROR: savepoint "s1" does not exist. O RELEASE **descarta o marcador** (não confirma nada no disco!), e o erro abortou a transação.

```
ROLLBACK;  
SELECT id, titular, saldo FROM conta ORDER BY id;
```

Resultado esperado: os mesmos saldos da seção 2.4. O ROLLBACK sem nome desfaz **a transação inteira**, com todos os savepoints dentro dela.

Saia com \q.

Anote: por que o SELECT da seção 2.3 funcionou, se o da Atividade 1 (seção 1.4) falhou?

Atividade 3 — Isolamento: o que a outra sessão enxerga

Tempo: 15 minutos · **Terminais:** 2 (abas **Sessão 1** e **Sessão 2**)

Objetivo: ver que uma transação não enxerga o que outra ainda não confirmou, e que duas transações não alteram a mesma linha ao mesmo tempo.

Nesta atividade você vai alternar entre as abas. Cada bloco de comandos diz em qual aba ele vai.

3.1 — Preparar e abrir as duas sessões

Faça o preparo na **Sessão 1**. Depois, **nas duas abas**, entre no banco:

```
podman exec -it bd2-aula07-pg psql -U aluno -d banco
```

3.2 — Uma alteração ainda não confirmada

Sessão 1:

```
BEGIN;  
UPDATE conta SET saldo = saldo - 300 WHERE id = 1;  
SELECT id, titular, saldo FROM conta WHERE id = 1;
```

Resultado esperado: Ana 700.00 — a Sessão 1 enxerga a própria alteração.

Sessão 2:

```
SELECT id, titular, saldo FROM conta WHERE id = 1;
```

Resultado esperado: Ana 1000.00. A Sessão 2 **não vê** o que a Sessão 1 ainda não confirmou. Não existe “leitura suja” no PostgreSQL.

3.3 — Duas transações querendo a mesma linha

Sessão 2:

```
UPDATE conta SET saldo = saldo + 1 WHERE id = 1;
```

Resultado esperado: nada acontece. O comando fica parado, sem devolver o prompt. A linha da Ana está reservada pela transação da Sessão 1, e a Sessão 2 entrou na fila.

Sessão 1:

```
COMMIT;
```

Resultado esperado: assim que a Sessão 1 confirma, a **Sessão 2** destrava sozinha e mostra UPDATE 1.

Sessão 2:

```
SELECT id, titular, saldo FROM conta WHERE id = 1;
```

Resultado esperado: Ana 701.00 — o débito de R\$ 300,00 da Sessão 1 e o crédito de R\$ 1,00 da Sessão 2, **um depois do outro**, sem que um apague o outro.

Saia com \q nas duas abas.

Anote: o que teria acontecido com o saldo da Ana se a Sessão 2 não precisasse esperar? (Esse é o tema da Aula 08: concorrência e níveis de isolamento.)

Atividade 4 — Durabilidade: derrubar o servidor

Tempo: 15 minutos · **Terminais:** 2

Objetivo: simular uma queda de energia e confirmar que o que foi confirmado sobrevive, e o que não foi confirmado some.

4.1 — Preparar e entrar no banco

Faça o preparo na **Sessão 1** e entre no banco:

```
podman exec -it bd2-aula07-pg psql -U aluno -d banco
```

4.2 — Uma transação confirmada e outra em aberto

Sessão 1:

```
-- confirmada: Davi recebe R$ 500,00
BEGIN;
UPDATE conta SET saldo = saldo + 500 WHERE id = 4;
COMMIT;

-- em aberto: alguém zera a conta do Bruno e NÃO confirma
BEGIN;
UPDATE conta SET saldo = 0 WHERE id = 2;
SELECT id, titular, saldo FROM conta ORDER BY id;
```

Resultado esperado: Bruno 0.00 e Davi 550.00 — na visão da Sessão 1.

4.3 — A queda

Na **Sessão 2** (no terminal, **fora** do psql), derrube o servidor como se a energia tivesse acabado:

```
podman kill bd2-aula07-pg
```

Resultado esperado: o comando imprime bd2-aula07-pg, e a **Sessão 1** volta sozinha para o prompt do terminal: o psql rodava dentro do contêiner e morreu com ele.

podman kill não pede licença: é o equivalente a puxar o cabo da tomada. O podman stop, ao contrário, avisa o PostgreSQL e espera ele fechar tudo com calma.

4.4 — Religar e ler o diário de bordo

Sessão 2:

```
podman start bd2-aula07-pg
podman logs --tail 10 bd2-aula07-pg
```

Resultado esperado: entre as últimas linhas do log aparecem:

```
LOG: database system was interrupted; last known up at ...
LOG: database system was not properly shut down; automatic recovery in progress
LOG: redo starts at ...
LOG: redo done at ...
LOG: database system is ready to accept connections
```

O PostgreSQL percebeu que não foi desligado direito e **refez** (*redo*) o que estava no **WAL**, o diário que ele grava antes de mexer nos dados.

Se aparecer também uma linha FATAL: the database system is starting up, não se preocupe: é uma conexão que chegou enquanto a recuperação acontecia.

4.5 — O que sobreviveu

Sessão 1:

```
podman exec -it bd2-aula07-pg psql -U aluno -d banco
```

```
SELECT id, titular, saldo FROM conta ORDER BY id;
```

Resultado esperado: Davi 550.00 (confirmado: **sobreviveu**) e Bruno 500.00 (não confirmado: **sumiu**). Isso é a **durabilidade** — e, de brinde, a atomicidade: a transação interrompida não deixou pedaço nenhum para trás.

Saia com \q.

Anote: as linhas do log que mostram a recuperação.

Atividade 5 — 2PC: as duas fases e o voto “não”

Tempo: 20 minutos · **Terminais:** 2

Objetivo: fazer uma transferência entre dois bancos de dados diferentes usando o protocolo de confirmação em duas fases, **com você no papel de coordenador**.

O cenário. A conta da Ana está no banco da **agência de Palmas**; a do Davi, no banco da **agência de Gurupi**. Um único `BEGIN ... COMMIT` não abrange os dois bancos. Se Palmas confirmar o débito e Gurupi falhar no crédito, o dinheiro some. O 2PC resolve isso em duas fases:

1. **Preparação:** cada participante faz a sua parte e **promete** que consegue confirmá-la (`PREPARE TRANSACTION`). Quem não consegue vota “não”.
2. **Decisão:** se **todos** votaram “sim”, o coordenador manda confirmar (`COMMIT PREPARED`); se **alguém** votou “não”, manda desfazer (`ROLLBACK PREPARED`).

5.1 — Preparar e abrir as duas agências

Faça o preparo na **Sessão 1**. Depois:

Sessão 1 — agência de Palmas:

```
podman exec -it bd2-aula07-pg psql -U aluno -d agencia_palmas
```

Sessão 2 — agência de Gurupi:

```
podman exec -it bd2-aula07-pg psql -U aluno -d agencia_gurupi
```

Em cada aba:

```
SELECT * FROM conta ORDER BY id;
```

Resultado esperado: Palmas mostra Ana 1000.00 e Bruno 500.00; Gurupi mostra Carla 200.00 e Davi 50.00.

5.2 — Fase 1: preparação (Ana → Davi, R\$ 300,00)

Sessão 1 (Palmas):

```
BEGIN;  
UPDATE conta SET saldo = saldo - 300 WHERE id = 1;  
PREPARE TRANSACTION 'transf1-palmas';
```

Sessão 2 (Gurupi):

```
BEGIN;  
UPDATE conta SET saldo = saldo + 300 WHERE id = 4;  
PREPARE TRANSACTION 'transf1-gurupi';
```

Resultado esperado: as duas respondem `PREPARE TRANSACTION`. Os dois participantes votaram “**sim**”.

Os nomes são diferentes (`transf1-palmas`, `transf1-gurupi`) porque, neste laboratório, as duas “agências” moram no mesmo servidor, e o PostgreSQL exige um nome único por servidor. Na vida real cada agência teria o seu servidor.

Sessão 1 (Palmas):

```
SELECT gid, database FROM pg_prepared_xacts ORDER BY gid;  
SELECT * FROM conta ORDER BY id;
```

Resultado esperado: duas transações preparadas, uma em cada banco. E Ana **ainda** aparece com `1000.00`: preparada não é confirmada.

5.3 — Fase 2: decisão “confirmar”

Todos votaram “sim”, então você, coordenador, manda confirmar.

Sessão 1 (Palmas):

```
COMMIT PREPARED 'transf1-palmas';  
SELECT * FROM conta ORDER BY id;
```

Sessão 2 (Gurupi):

```
COMMIT PREPARED 'transf1-gurupi';  
SELECT * FROM conta ORDER BY id;
```

Resultado esperado: `COMMIT PREPARED` nas duas; Ana `700.00` em Palmas e Davi `350.00` em Gurupi.

5.4 — Um participante vota “não” (Bruno → Carla, R\$ 900,00)

Sessão 1 (Palmas):

```
BEGIN;  
UPDATE conta SET saldo = saldo - 900 WHERE id = 2;
```

Resultado esperado: Bruno só tem R\$ 500,00 — erro de `CHECK` (`conta_saldo_check`). Palmas **não consegue** cumprir a sua parte.

```
PREPARE TRANSACTION 'transf2-palmas';
```

Resultado esperado: o servidor responde **ROLLBACK**. Uma transação abortada não pode ser preparada: este é o voto “**não**”.

Sessão 2 (Gurupi):

```
BEGIN;  
UPDATE conta SET saldo = saldo + 900 WHERE id = 3;  
PREPARE TRANSACTION 'transf2-gurupi';
```

Resultado esperado: PREPARE TRANSACTION — Gurupi votou “sim”, sem saber do problema em Palmas.

5.5 — Fase 2: decisão “desfazer”

Um voto “nã” basta para cancelar tudo. **Sessão 1 (Palmas):**

```
SELECT gid, database FROM pg_prepared_xacts;  
COMMIT PREPARED 'transf2-palmas';
```

Resultado esperado: só transf2-gurupi aparece na lista; e o COMMIT PREPARED falha com ERROR: prepared transaction with identifier "transf2-palmas" does not exist — não há o que confirmar em Palmas.

Sessão 2 (Gurupi):

```
ROLLBACK PREPARED 'transf2-gurupi';  
SELECT * FROM conta ORDER BY id;
```

Resultado esperado: ROLLBACK PREPARED; Carla continua com 200.00. Ninguém recebeu dinheiro que não saiu de lugar nenhum.

Saia com \q nas duas abas.

Anote: por que o coordenador não pode confirmar em Gurupi só porque Gurupi votou “sim”?

Atividade 6 — 2PC: o coordenador caiu

Tempo: 15 minutos · **Terminais:** 2

Objetivo: ver o ponto fraco do 2PC — o participante que votou “sim” fica preso à promessa até o coordenador decidir, mesmo que o servidor caia.

6.1 — Preparar e abrir as duas agências

Faça o preparo na **Sessão 1**. Depois abra Palmas na **Sessão 1** e Gurupi na **Sessão 2**, como na seção 5.1.

6.2 — Fase 1 completa (Bruno → Carla, R\$ 100,00)

Sessão 1 (Palmas):

```
BEGIN;  
UPDATE conta SET saldo = saldo - 100 WHERE id = 2;  
PREPARE TRANSACTION 'transf3-palmas';
```

Sessão 2 (Gurupi):

```
BEGIN;  
UPDATE conta SET saldo = saldo + 100 WHERE id = 3;  
PREPARE TRANSACTION 'transf3-gurupi';
```

Resultado esperado: PREPARE TRANSACTION nas duas. Os dois votaram “sim”... e o coordenador “cai” antes de mandar a decisão.

6.3 — A queda

Saia do psql na **Sessão 2** (\q) e derrube o servidor:

```
podman kill bd2-aula07-pg  
podman start bd2-aula07-pg  
podman logs --tail 12 bd2-aula07-pg
```

Resultado esperado: além das linhas de recuperação da Atividade 4, aparecem duas linhas recovering prepared transaction ... from shared memory e a linha 2 two-phase state files were written for long-running prepared transactions.

6.4 — A promessa sobreviveu

Sessão 1 (que voltou ao terminal com a queda) — entre de novo em Palmas:

```
podman exec -it bd2-aula07-pg psql -U aluno -d agencia_palmas
```

```
SELECT gid, database, prepared FROM pg_prepared_xacts ORDER BY gid;  
SELECT * FROM conta ORDER BY id;
```

Resultado esperado: as duas transações preparadas **continuam lá** depois da queda. E Bruno ainda aparece com 500.00: a decisão não foi tomada.

6.5 — A linha presa

Tente mexer na conta do Bruno. Para não ficar esperando para sempre, limite a espera a três segundos:

```
SET lock_timeout = '3s';
UPDATE conta SET saldo = saldo + 10 WHERE id = 2;
```

Resultado esperado: depois de três segundos:

```
ERROR: canceling statement due to lock timeout
```

A linha do Bruno está **bloqueada** pela transação preparada. Ninguém consegue alterá-la até alguém decidir o destino da transferência. É o chamado **problema do bloqueio** do 2PC.

6.6 — Alguém decide

O coordenador “voltou” (é você). Tente decidir a parte de Gurupi estando em Palmas:

```
COMMIT PREPARED 'transf3-gurupi';
```

Resultado esperado:

```
ERROR: prepared transaction belongs to another database
HINT: Connect to the database where the transaction was prepared to finish it.
```

Cada participante decide a própria parte. Confirme em cada um:

Sessão 1 (Palmas):

```
COMMIT PREPARED 'transf3-palmas';
SELECT * FROM conta ORDER BY id;
```

Sessão 2 (Gurupi):

```
podman exec -it bd2-aula07-pg psql -U aluno -d agencia_gurupi
```

```
COMMIT PREPARED 'transf3-gurupi';
SELECT * FROM conta ORDER BY id;
```

Resultado esperado: Bruno 400.00 em Palmas e Carla 300.00 em Gurupi.

Saia com \q nas duas abas.

Anote: o que aconteceria com a conta do Bruno se o coordenador nunca mais voltasse?

Atividade 7 — A transação pelo pgAdmin

Tempo: 15 minutos · **Terminais:** 1 + navegador

Objetivo: controlar uma transação numa ferramenta gráfica e perceber que o pgAdmin também trabalha, por padrão, em *autocommit*.

7.1 — Preparar e abrir o pgAdmin

Faça o preparo na **Sessão 1**. Depois clique na aba **pgAdmin**; ela abre uma janela nova. O pgAdmin leva cerca de trinta segundos para subir: se a página não abrir, espere e recarregue.

Entre com:

Campo	Valor
Email Address / Username	aluno@bd2.unitins.br
Password	aluno

7.2 — Conectar ao servidor

1. No painel **Object Explorer**, à esquerda, clique duas vezes em **Banco de Dados II**.
2. Aparece a janela **Connect to Server**: digite a senha do banco, aluno, e clique em **OK**.
3. Abra **Databases** e clique uma vez em **banco**.
4. No menu **Tools**, escolha **Query Tool**.

Resultado esperado: abre-se uma aba chamada banco/aluno@BD2 - Aula 07 (banco), com um editor de SQL.

7.3 — Desligar o *autocommit*

Na barra de botões do Query Tool, ao lado dos botões de executar (▶), há uma **setinha para baixo** (*Execute options*). Clique nela e **desmarque Auto commit?**.

7.4 — Uma alteração pendente

Digite no editor e tecla **F5** (*Execute script*):

```
UPDATE conta SET saldo = saldo - 100 WHERE id = 1;
```

Resultado esperado: na aba **Messages**, UPDATE 1. O ícone à esquerda do nome da conexão vira um **relógio**; passando o mouse sobre ele aparece *The session is idle in a valid transaction block*: há uma transação aberta.

Agora olhe de fora. Na **Sessão 1** (terminal):

```
podman exec bd2-aula07-pg psql -U aluno -d banco -c "SELECT id, titular, saldo
FROM conta WHERE id = 1;"
```

Resultado esperado: Ana 1000.00 — o terminal não vê a alteração pendente.

7.5 — Desfazer e confirmar pelos botões

Ainda na barra do Query Tool há dois botões com o desenho de um banco de dados: um com ✓ (**Commit**) e um com uma seta circular (**Rollback**). Passe o mouse sobre eles para ver o nome.

1. Clique em **Rollback** e repita a consulta no terminal: Ana 1000.00.
2. Execute o UPDATE de novo (**F5**), clique em **Commit** e repita a consulta no terminal.

Resultado esperado: agora Ana aparece com 900.00, e o ícone da conexão volta ao normal (*The session is idle and there is no current transaction*).

7.6 — Para ver sozinho (opcional)

Volte à setinha *Execute options* e **marque de novo Auto commit?** (é o padrão de toda aba nova do Query Tool). Escreva no editor o bloco completo e execute com **F5**:

```
BEGIN;
UPDATE conta SET saldo = saldo - 50 WHERE id = 2;
UPDATE conta SET saldo = saldo + 50 WHERE id = 3;
COMMIT;
```

O pgAdmin respeita o BEGIN/COMMIT que você escreve, exatamente como o psql.

Anote: desligue de novo o *autocommit*, execute um UPDATE e feche a aba do Query Tool. O pgAdmin pergunta primeiro se deve salvar o texto da consulta (responda *Don't save*) e depois mostra *Commit transaction?*. Escolha **Rollback** e confira o saldo no terminal: o que aconteceu, e por quê?

No seu computador, com o Podman Desktop

O mesmo ambiente roda no seu computador. Baixe os três arquivos para uma pasta vazia:

- <https://raw.githubusercontent.com/alyssonbruno/publicacao/main/BD2/AULA07/compose.yaml>

- <https://raw.githubusercontent.com/alyssonbruno/publicacao/main/BD2/AULA07/preparar.sql>
- <https://raw.githubusercontent.com/alyssonbruno/publicacao/main/BD2/AULA07/servers.json>

Abra um terminal nessa pasta e rode `podman compose up -d`. O pgAdmin fica em <http://localhost:8181>. Em vez de uma segunda aba, abra um segundo terminal.

No **PowerShell** do Windows, o `<` não funciona. Use no lugar do preparo:

```
podman exec bd2-aula07-pg psql -U aluno -d banco -f /docker-entrypoint-initdb.d/01-preparar.sql
```

Esse comando funciona em qualquer sistema, porque o `compose.yaml` já coloca o `preparar.sql` dentro do contêiner.

Ao terminar: `podman compose down` desliga e guarda os dados; `podman compose down -v` desliga e apaga tudo.

Se algo der errado

Sintoma	Causa provável	O que fazer
connection refused ou the database system is starting up	o servidor ainda está subindo	espere dez segundos e repita
ERROR: current transaction is aborted	um erro anterior abortou a transação	ROLLBACK; e recomece o trecho
um comando fica parado, sem devolver o prompt	outra sessão (ou uma transação preparada) segura a linha	confirme ou desfça a transação na outra aba; em último caso, rode o preparo de novo
prepared transactions are disabled	o PostgreSQL não foi iniciado pelo <code>compose.yaml</code> da aula	suba o ambiente com <code>podman compose up -d</code> na pasta <code>~/lab07</code>
Error: no container with name or ID "bd2-aula07-pg"	o ambiente não foi iniciado	faça o preparo
a aba pgAdmin não abre	o pgAdmin ainda está subindo	espere trinta segundos e recarregue
deu tudo errado	—	rode o preparo de novo: ele encerra as sessões, desfaz transações preparadas e recria as contas

Checklist

- ☐ Vi o COMMIT de uma transação abortada responder ROLLBACK (Atividade 1).
- ☐ Usei SAVEPOINT para desfazer só um pagamento do lote (Atividade 2).
- ☐ Vi uma sessão não enxergar o que a outra não confirmou (Atividade 3).
- ☐ Derrubei o servidor e vi o WAL recuperar o que foi confirmado (Atividade 4).
- ☐ Fiz uma transferência entre dois bancos com PREPARE TRANSACTION e COMMIT PREPARED (Atividade 5).

- ☐ Vi uma transação preparada sobreviver à queda e segurar uma linha (Atividade 6).
- ☐ Controlei uma transação pelos botões do pgAdmin (Atividade 7).
- ☐ Respondi ao questionário da aula no Google Classroom.